



Antes de empezar...

- Dudas de tarea 1
- Examen 1, Octubre 1

Inteligencia Artificial

Clase 5: Satisfacción de restricciones



Para el día de hoy

- Problemas de satisfacción de restricciones



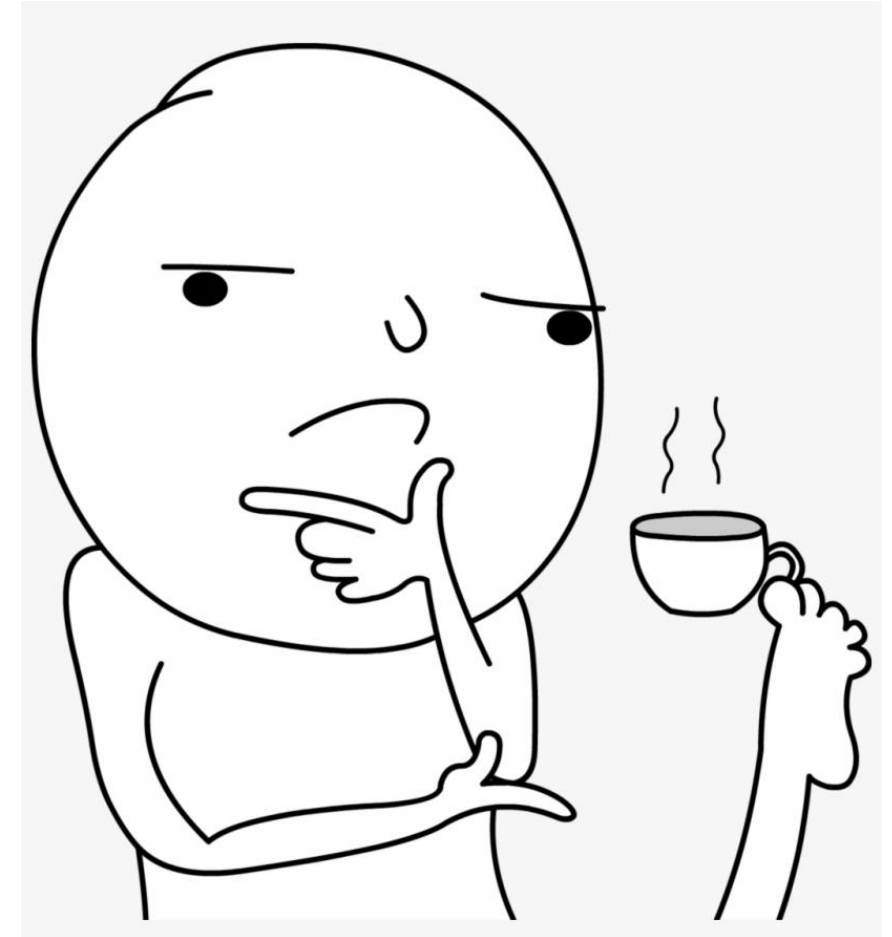
Ahora relajémonos y resolvamos un sudoku...

- ¿Búsqueda en anchura?
- ¿Búsqueda en profundidad?
- ¿A*?

	2			8	9		4	7
		9	7		4			
	6	4				9		8
	5	3	9	7	8			6
		6		4		7	9	
		7		3		2		5
6		1	5	9		8	2	4
5		2	8		6	3		9
	3	8			7	5	6	1

Hmmm...

- Hemos tratado con las suposiciones que nuestro mundo es:
 - De un solo agente
 - Acciones deterministas
 - Totalmente observable
 - Espacio de estados discreto
- Hemos planeado una secuencia de opciones
 - El camino a la meta es lo más importante
 - Los caminos tienen costos y profundidades
 - Las heurísticas dan guía para problemas específicos
- Pero en sudoku...
 - La meta es lo importante (llenar el tablero)
 - Todos los caminos son de la misma profundidad



Problemas de satisfacción de restricciones

- Un subconjunto de los problemas de búsqueda
- Los estados se definen por variables X_i con valores de un dominio D
- La prueba de la meta es un conjunto de restricciones especificando la combinación de valores para subconjuntos de las variables



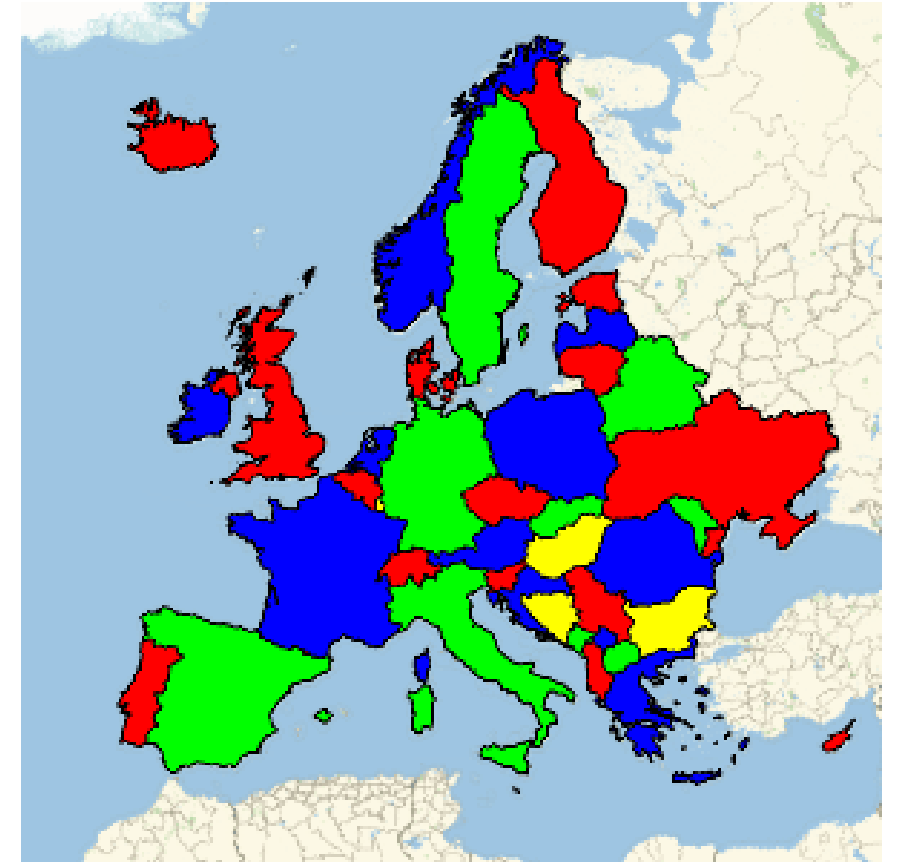
Ejemplo: sudoku

- Variables:
 - Cada espacio abierto
- Dominio:
 - $\{1, \dots, 9\}$
- Restricciones:
 - Cada columna debe tener un valor diferente del 1 al 9
 - Cada fila debe tener un valor diferente del 1 al 9
 - Cada región de 9 debe tener un valor diferente del 1 al 9

	2			8	9		4	7
		9	7		4			
	6	4				9		8
	5	3	9	7	8			6
		6		4		7	9	
		7		3		2		5
6		1	5	9		8	2	4
5		2	8		6	3		9
	3	8			7	5	6	1

Ejemplo: coloreado de mapas

- Variables: ciudades/países/etc.
- Dominios: colores
- Restricciones
 - Implícitas: $estado_1 \neq estado_2$
 - Explícitas: $estado_1 \in verde$
- La solución son asignaciones que satisfacen todas las restricciones



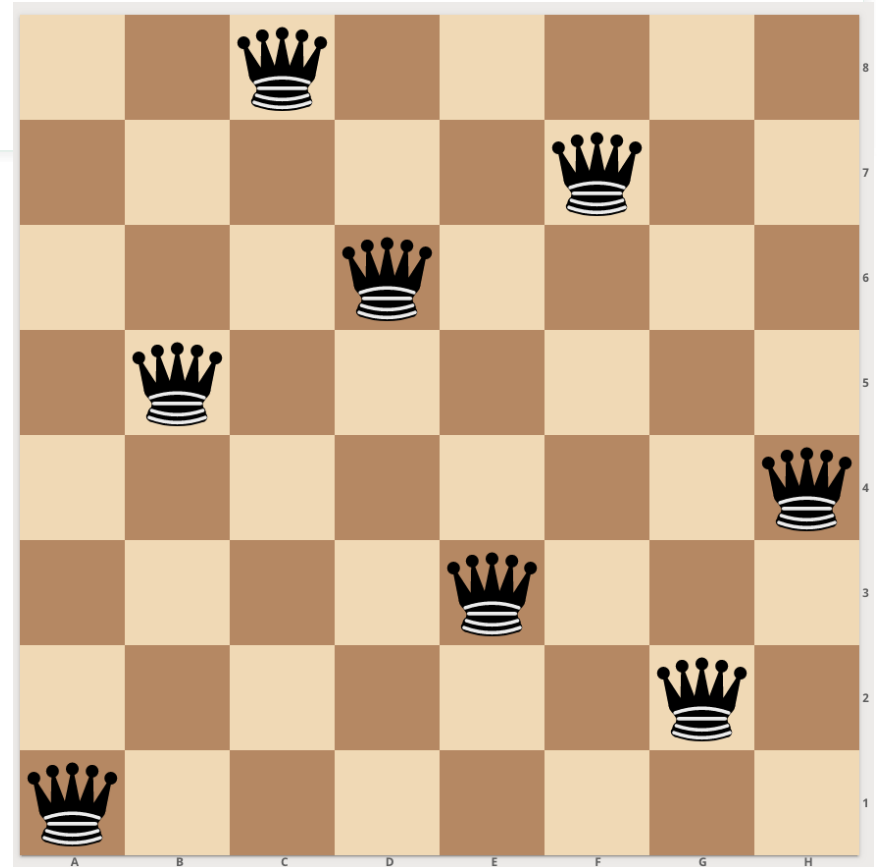
Ejemplo: Criptoaritmetica

- Variables:
- Dominio:
- Restricciones:

$$\begin{array}{r} \text{TWO} \\ + \text{TWO} \\ \hline \text{FOUR} \end{array}$$

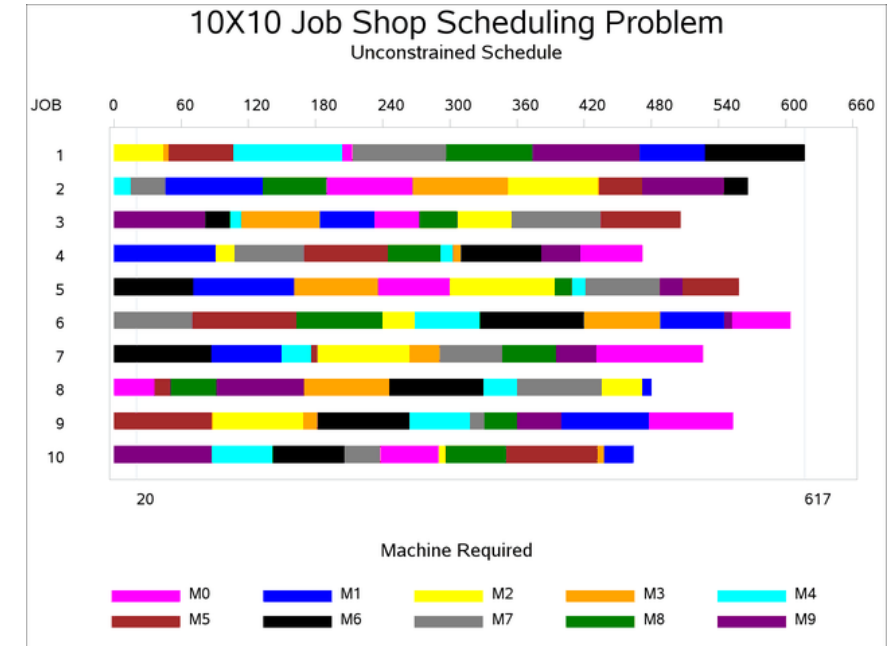
Ejemplo: N-reinas

- Variables:
- Dominio:
- Restricciones:



Tipos de problemas de satisfacción de restricciones (CSP)

- Variables discretas
 - Dominios finitos: $O(d^n)$ para d opciones y n variables (satisfactibilidad booleana)
 - Dominios infinitos: enteros, cadenas, etc. (calendarización de trabajos)
- Variables continuas
 - Problemas lineales (programación/optimización lineal)
 - Problemas no lineales (programación/optimización no lineal)



Tipos de restricciones

- Fuertes
 - Unarias
 - $ciudad_1 \neq azul$
 - Binarias
 - $ciudad_1 \neq ciudad_2$
 - De alto orden
 - Todos deben ser diferentes
- Suaves
 - Rojo es mejor que verde
 - Representan el costo de la asignación
 - Le dan problemas a CSPs



Aplicaciones

- Problemas de calendarización
- Problemas de tiempos
- Problemas de asignación
- Transporte
- Calendarización en fábricas
- Etc. etc. etc.

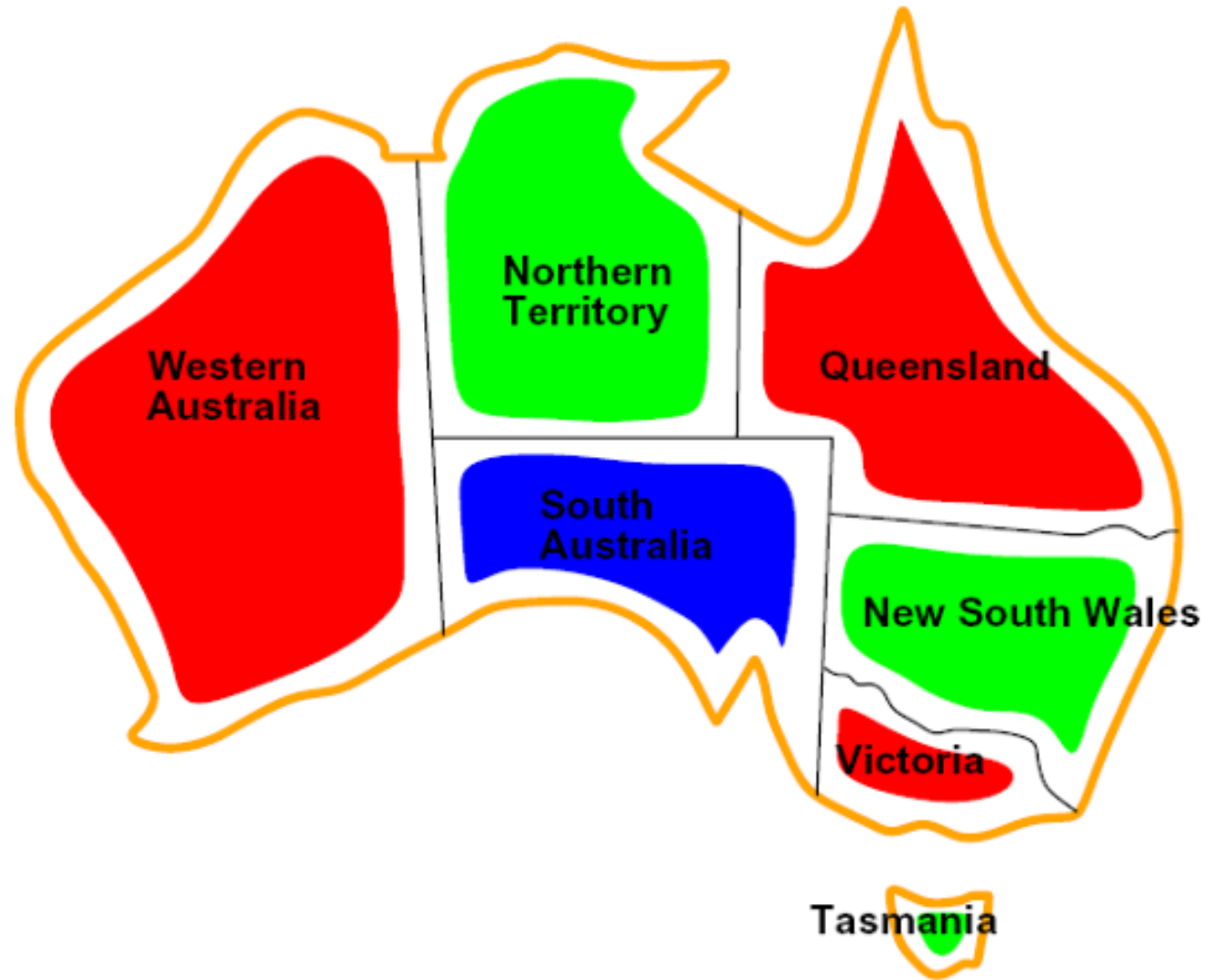


Formulando un CSP

- Estados definidos por los valores asignado hasta ahora
 - Estado inicial: {}
 - Función sucesor: asignar un valor a una variable no asignada
 - Prueba de meta: la asignación esta completa y satisface todas las restricciones



Nuestro ejemplo

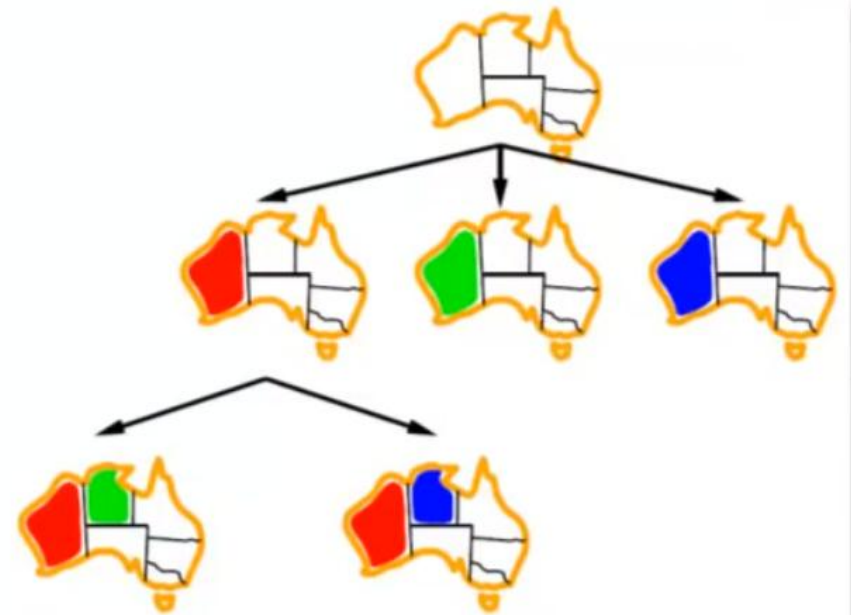


A collection of school supplies is arranged on a dark green background. In the top left, a portion of a white calculator is visible with buttons for percentage, MU, equals, and plus. Below it is a blue folder with the name 'Kaill xian' written on it. A blue binder clip is attached to a silver ruler. A compass, a pair of scissors, a yellow paper cutter, and a yellow sharpener are also present. A small orange horizontal bar is located in the top right corner.

Las nuevas herramientas

Backtracking

- Una variable a la vez
 - Fijar el orden dado que las variables son conmutativas
 - En cada paso solo se fija una variable
- Revisar las restricciones en cada paso
 - Considerar valores que no tienen conflicto con asignaciones previas
- Búsqueda en profundidad con estos elementos se llama búsqueda con backtracking
- Puede resolver el problema de n-reinas para $n \approx 25$



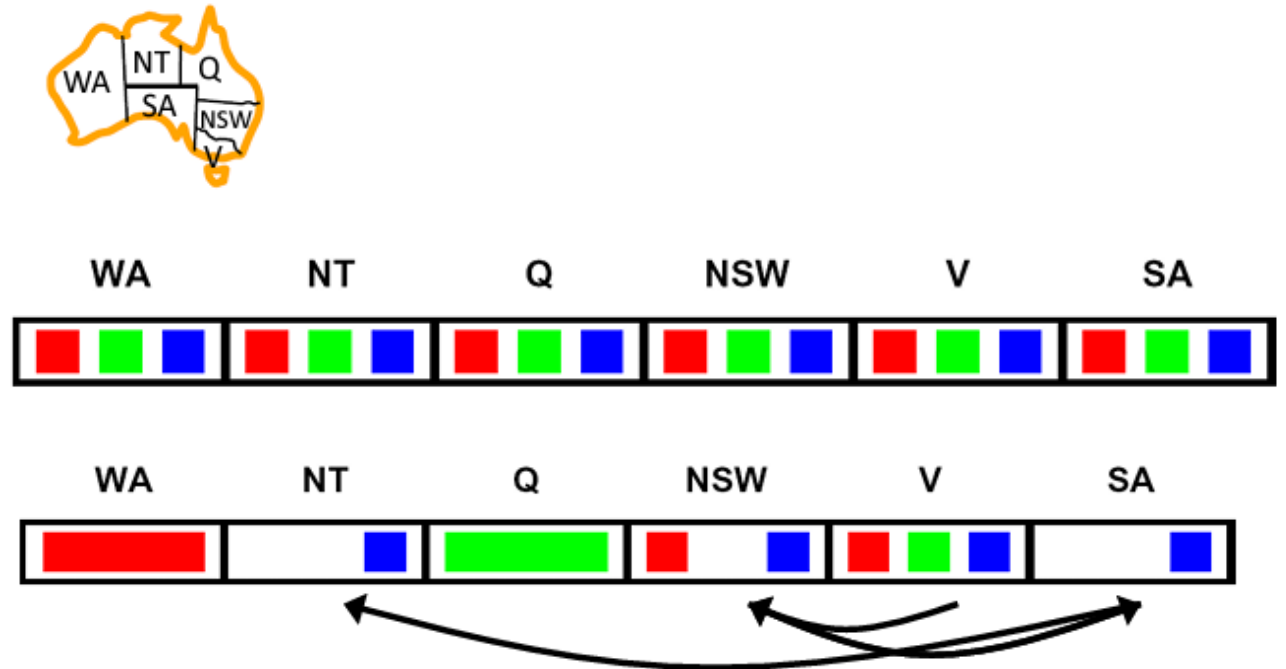
Mejorando backtracking

- ¿Cuál variable debemos trabajar?
- ¿En qué orden debemos intentar?
- ¿Podemos detectar fallos rápido?
- ¿Podemos explotar la estructura del problema?



Filtrado

- Mantener un registro de los dominios de cada variable y eliminar malas opciones
- Revisión hacia adelante: Quitar valores que violan alguna restricción cuando son añadidos a una asignación
- Consistencia (Propagación de restricciones): un arco $X \rightarrow Y$ es consistente si y solo si para cada x en la cola existe algún y en la cabeza que puede ser asignado sin violar una restricción



Un algoritmo para filtrado

function AC-3(*csp*) **returns** the CSP, possibly with reduced domains

inputs: *csp*, a binary CSP with variables $\{X_1, X_2, \dots, X_n\}$

local variables: *queue*, a queue of arcs, initially all the arcs in *csp*

while *queue* is not empty **do**

$(X_i, X_j) \leftarrow \text{REMOVE-FIRST}(\textit{queue})$

if REMOVE-INCONSISTENT-VALUES(X_i, X_j) **then**

for each X_k **in** NEIGHBORS[X_i] **do**

 add (X_k, X_i) to *queue*

function REMOVE-INCONSISTENT-VALUES(X_i, X_j) **returns** true iff succeeds

removed $\leftarrow$ false

for each x **in** DOMAIN[X_i] **do**

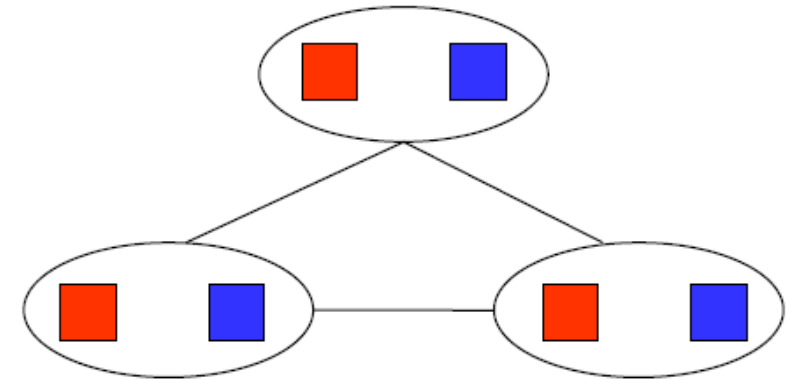
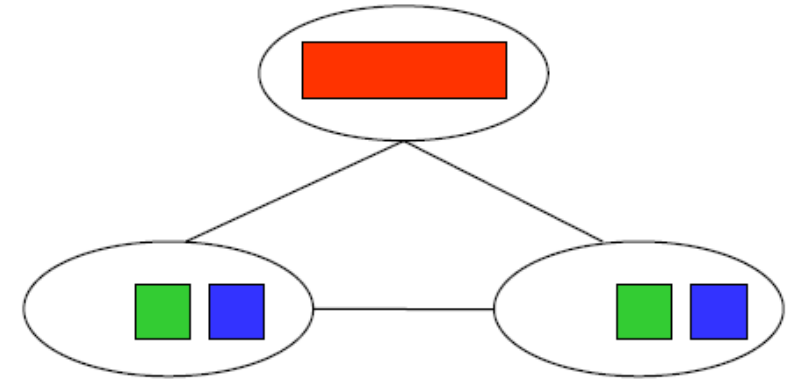
if no value y in DOMAIN[X_j] allows (x, y) to satisfy the constraint $X_i \leftrightarrow X_j$

then delete x from DOMAIN[X_i]; *removed* $\leftarrow$ true

return *removed*

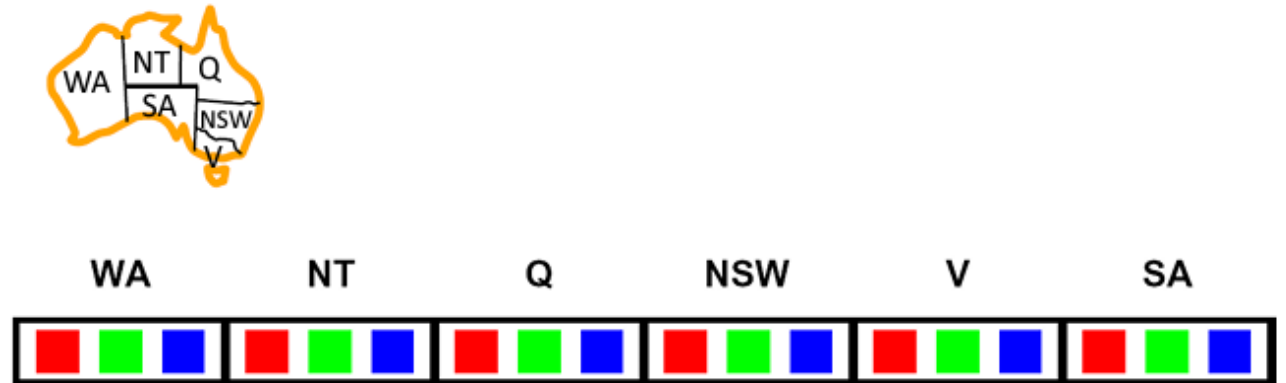
Limitaciones de consistencia

- Después de la revisión de consistencia podemos tener:
 - Una sola solución
 - Múltiples soluciones
 - No tener soluciones y no saberlo
- La consistencia es parte de búsqueda con backtracking



Ordenamiento

- Elegir la variable con más restricciones
- Es decir, el lugar donde hay menos valores disponibles, o
- Elegir el valor con menos restricciones, aquella que quita menos valores a las otras variables
- ¿Por qué?



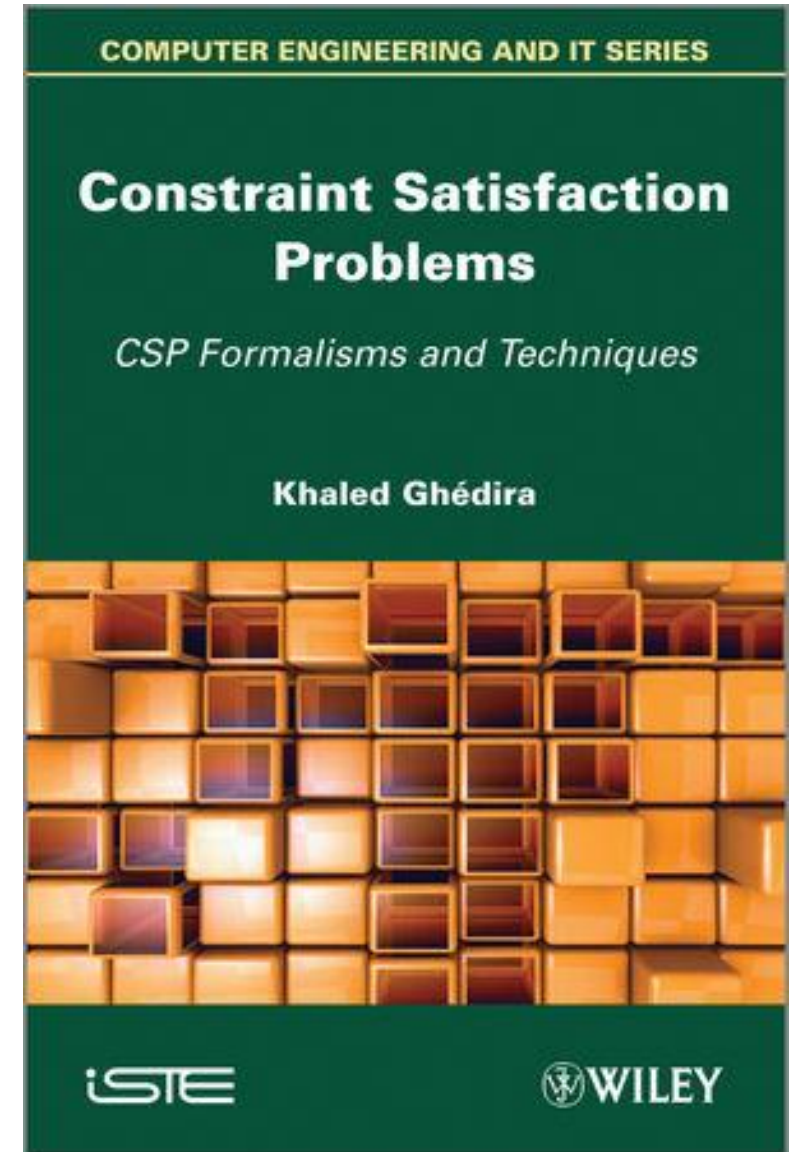
El algoritmo

```
function BACKTRACKING-SEARCH(csp) returns solution/failure
  return RECURSIVE-BACKTRACKING({ }, csp)

function RECURSIVE-BACKTRACKING(assignment, csp) returns soln/failure
  if assignment is complete then return assignment
  var ← SELECT-UNASSIGNED-VARIABLE(VARIABLES[csp], assignment, csp)
  for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
    if value is consistent with assignment given CONSTRAINTS[csp] then
      add {var = value} to assignment
      result ← RECURSIVE-BACKTRACKING(assignment, csp)
      if result ≠ failure then return result
      remove {var = value} from assignment
  return failure
```

Para saber más

- Software y bibliotecas
 - Minizinc
 - OR Tools
 - CPLEX
 - Gurobi
 - Gecode
 - Chuffed





Ahora si...
resolvamos el
sudoku

	2			8	9		4	7
		9	7		4			
	6	4				9		8
	5	3	9	7	8			6
		6		4		7	9	
		7		3		2		5
6		1	5	9		8	2	4
5		2	8		6	3		9
	3	8			7	5	6	1

Para la otra vez...

- Búsqueda local
- Búsqueda en espacios continuos



The End.